Interview Questions and Answers

MedInfo Assistant - Medical Research & Report Analysis Multi-Agent GenAI Platform

Generated 2026-10-15 from 13-INTERVIEW-QA.md - information only, not medical advice.

Contents

1. Project overview
2. Architecture
3. Multi-agent design
4. RAG and retrieval
5. Ingestion, OCR and data
6. Medical safety
7. Security and privacy
8. LLM providers and resilience
9. Evaluation and testing
10. Observability and operations
11. Delivery and deployment
12. Trade-offs and judgement
13. Curveball questions
14. Two-minute walkthrough (for "tell me about a project you built")

Likely questions from technical interviewers, portfolio reviewers and stakeholders, with model answers based on what this project actually implements. Answers distinguish **implemented and tested offline**, **verified live** (real Supabase + EURI, 2026-10-14) and **planned**. Numbers are from the latest local runs: 419 backend tests (240 in the safety suite), 10 browser tests.

---

1. Project overview

Q1. Describe the project in 30 seconds. A safety-first medical information assistant. It answers only from trusted, cited sources, leads with emergency advice when symptoms are urgent, and never diagnoses or prescribes. Under the hood it is a fixed pipeline of specialised agents - guardrails, triage, scope classification, hybrid retrieval with re-ranking, a grounded answer agent and a verifier - built with FastAPI, LangGraph, Supabase Postgres/pgvector and OpenAI-compatible models, with a plain-JS frontend.

Q2. What problem does it solve? General chatbots answer from memory, rarely cite sources, can be confidently wrong, may diagnose or suggest doses, and do not reliably escalate emergencies. Uploaded reports and web pages can also contain hidden instructions and personal health data. This product answers only from traceable evidence, escalates emergencies first, refuses unsafe clinical decisions and protects PHI.

Q3. Who are the users? Public users (plain-language information), clinicians (research mode with allow-listed web sources and a review queue) and admins (curate the trusted corpus, see evaluation). Roles come only from Supabase `app_metadata`.

Q4. What is out of scope, deliberately? Diagnosis, prescribing, choosing a medicine or changing a dose, replacing a clinician, and any claim of clinical validation or regulatory approval. Real patient data is not processed until governance approvals (DPIA, provider DPAs, clinical safety case) are in place.

Q5. What is the current status, honestly? All features are implemented and covered by offline tests. The core path is verified live: Supabase schema, ingestion of 11 synthetic documents (25 chunks, 2 via OCR), hybrid retrieval with re-ranking, cited answers, the emergency path, "not covered", and sign-in for all three roles. The live evaluation ran on 2026-10-15: the safety suite passed 100% live, and two dosing/interaction questions were over-refused. Not yet run: research mode with real Tavily, live Gemini/Groq fallback, Docker builds, the CI pipeline, and any AWS deployment (Terraform is validated only).

---

2. Architecture

Q6. Walk me through the architecture. Browser -> nginx frontend (static UI, strict CSP, proxies `/api` with SSE unbuffered) -> FastAPI backend. Inside the backend, `ChatService` runs the pipeline: input guardrails -> triage -> scope -> answer cache -> LangGraph graph (rewrite -> retrieve -> generate) -> output guardrails -> verifier -> fixed safety wording. Data lives in Supabase Postgres (pgvector for chunks, plus app tables); Redis holds caches, short-term memory and rate-limit counters; models are reached through an OpenAI-compatible fallback chain.

Q7. Why a fixed pipeline instead of autonomous agents that collaborate freely? Predictability and auditability. In medicine I need to guarantee that triage always runs first and that every answer is verified. Free-form agent negotiation is non-deterministic and hard to test. I kept autonomy only where it adds value - the research agent chooses which tools to call - and even its output goes through the same verification.

Q8. How is the code organised? Ports and adapters. Use cases depend on interfaces (`ChatModel` , `Embedder` , `VectorIndex` , `DocumentRegistry` , `KeyValueStore` ...). Provider SDKs are imported only in `providers.py` and `container.py` (the composition root). Every adapter has an offline fake, so the whole pipeline - including the safety suite - runs in CI without credentials.

Q9. Why FastAPI with sync handlers? The provider SDK calls are blocking; sync handlers run in the server's thread pool, which keeps the code simple. I scale by replicas with one uvicorn worker per container, which also suits Kubernetes.

Q10. How does streaming work? `POST /api/chat/stream` returns server-sent events. The pipeline runs in a worker thread and pushes real phase events (safety -> searching -> generating -> verifying) from LangGraph node updates, then the final answer. The frontend parses SSE with `fetch` because `EventSource` cannot send POST bodies or auth headers.

---

3. Multi-agent design

Q11. What agents are there and what does each do? | Agent | Does | Cannot | |---|---|---| | Triage | Rules + temperature-0 LLM; emergency/crisis decision | Write the emergency text (built in code) | | Scope | 7 intent classes; stricter of rules/LLM wins | Relax a rules decision | | Query rewriter | Standalone queries for follow-ups | Change the answer | | Retriever + re-ranker | Hybrid search, RRF, LLM picks passages | Add content; fails open | | Answer agent | Cited sentences from numbered passages | Run without passages | | Verifier | Dose/diagnosis/citation/number/drug checks + judge | Pass what it cannot verify | | Research agent (ReAct) | Chooses corpus/web tools, max 3 calls | Write the answer; leave the allow-list |

Q12. How can a user see what the agents did? Every answer has a "How this answer was produced" panel: one row per agent with its decision, status and duration. It is built from decisions and counts only - never question, answer or passage text - and a release-blocking test proves no PHI ever appears in it.

Q13. How is the ReAct research agent controlled? It has two tools (`search_corpus` , `search_web`), at most three tool calls, temperature 0. It only gathers evidence; the answer is still written by the grounded answer agent and checked by the verifier. Before a model is used as an agent it must pass a real tool-call round trip. Gemini is excluded because its OpenAI-compatible API drops tool-call thought signatures. Status: implemented and tested with fakes; not yet run live.

Q14. Why rules plus an LLM for triage, instead of just an LLM? Defence in depth. Rules are deterministic, fast, local and testable; the LLM catches paraphrases the rules miss. Either one firing triggers the emergency notice, and if the LLM is down the rules still work.

---

4. RAG and retrieval

Q15. How does retrieval work? Hybrid search per collection: pgvector cosine similarity and a BM25-style keyword search inside Postgres (IDF from document frequencies over the filtered collection, on a generated `tsvector` column with a GIN index). The two rankings are merged by reciprocal rank fusion (k = 60); the top 12 candidates go to an LLM re-ranker that returns the relevant passage numbers.

Q16. Why hybrid instead of vectors only? Medical text has exact identifiers - drug names, codes, doses - where keyword search beats embeddings. I have a test where every embedding is identical and only keyword search can find the product code; RRF lets both signals contribute without comparing incompatible scores.

Q17. How do you handle conflicting guideline versions? Passages are grouped into a series (explicit `series` metadata or the title without version tokens); older versions are marked superseded and placed after the newest, the prompt tells the model to prefer the newest and say that versions differ, and the answer carries a code-generated warning. This was verified live: the hypertension answer cited both v3 and v2 and stated the conflict.

Q18. How do follow-up questions work? With history, a rewriter turns "and in pregnancy?" into a standalone query plus alternatives; all are searched and fused. On a first question it is skipped to save a model call.

Q19. What happens when nothing relevant is found? The answer model is not called at all and the user sees "not covered by the available sources". Verified live with a pancreatic-cancer question.

---

5. Ingestion, OCR and data

Q20. Which document types are supported? PDF (per page), DOCX, PPTX, XLSX, CSV/TSV, text/Markdown, PNG/JPG. Citations keep page, section, slide or sheet. Scanned pages go through RapidOCR (offline ONNX models).

Q21. Any OCR lessons? With default settings RapidOCR dropped whole lines and merged words on my test scans. I measured alternatives and set `det_unclip_ratio=2.0` plus grayscale/autocontrast/padding, which recovered every line with correct spacing. Verified live: the emergency answer cited the scanned chest-pain page.

Q22. Which chunking strategies? Recursive (1000 chars, 15% overlap), semantic (sentence-embedding breakpoints), parent-child (2000/400, children 10% overlap; parents returned for context) and table-aware chunking that never splits rows and repeats the header.

Q23. How is incremental ingestion done? Content hashing. Each chunk's hash covers text, location, provenance and chunker settings; chunk ids are position-independent, so inserting a paragraph re-embeds only what changed. Unchanged files embed nothing; stale chunks are deleted. CRLF is normalised before hashing. A real bug I found: re-saved DOCX files have new timestamps, so identical content looked "updated" - now content-level comparison reports "unchanged". Verified live: re-ingesting all 11 documents skipped all of them.

Q24. How are trusted sources separated from user uploads? Different collections per embedding model. Trusted ingestion is admin-only and requires a publisher on an allow-list with licence recorded. User uploads are untrusted, private to their owner (enforced in SQL on every query), labelled "user-provided" in citations, PHI-redacted and scanned for injected instructions before embedding.

Q25. What if you change the embedding model? Collection names encode provider, model and dimensions, so a new model gets new collections and a re-ingest. Vectors from different models are never mixed.

---

6. Medical safety

Q26. How do you prevent the system from diagnosing? Three layers: the scope agent routes "Do I have...?" to an educational mode with a referral; the prompt forbids personalised conclusions; and a deterministic detector removes sentences like "you probably have" or "this sounds like" (excluding conditionals such as "if you have"), adding a referral.

Q27. How do you prevent dosing advice? Any dose or start/stop/change wording must be a verbatim quote from a cited passage; otherwise the whole answer is replaced by a refusal plus "talk to your doctor or pharmacist". The safety suite includes an adversarial fake model that tries to output "Take 40 mg twice daily" - the safety layer still wins.

Q28. How do you prevent hallucination? Retrieval gate (no passages -> no model call), a grounded prompt, and code checks: every sentence must cite a provided passage, every number and known drug name must appear in the passages, and high-risk topics (dosing, pregnancy, children, interactions) need a temperature-0 faithfulness judge that fails closed. Failures replace the answer with a refusal.

Q29. How does emergency handling work? Triage runs on every question before anything else. If it fires, an emergency notice built in code with the configured number is placed first and pinned in the UI; self-harm adds crisis resources and stops generation. The notice survives provider outages and, after the safety review, also a stale conversation id, a memory outage, a rate limit, an over-long question or any pipeline crash (rules-only fallback, no model call).

Q30. What did the independent safety review find? A `safety-reviewer` agent found one high and several medium issues in my code. The worst: the model-written "limitations" line bypassed every post-answer check - a test showed a diagnosis, an unquoted dose and an email address passing through. Second: an unknown `conversation_id` returned 404 before triage ran, so an emergency got no notice. Both were reproduced, fixed and pinned with 17 release-blocking regression tests.

Q31. Are the emergency rules clinically approved? No. The red-flag file is marked `draft_pending_clinician_review` and needs a clinician's sign-off before real use. I am explicit about that in the docs and the UI copy.

---

7. Security and privacy

Q32. How is authentication done? Supabase email/password; the backend verifies JWTs against JWKS and accepts only ES256/RS256 (HS256 and "none" are rejected before key lookup), checks audience, issuer and expiry, and reads roles only from `app_metadata`. A live bug: tokens were rejected as "not yet valid" because the local clock was a few seconds behind Supabase; I added a 30-second leeway, with a test that far-future tokens are still rejected.

Q33. How do you protect PHI? Minimise and redact locally before anything leaves the server: names, MRN/IDs, dates of birth, phone, email, addresses, national IDs. Only redacted text reaches models, cache keys, memory and logs; stored messages are also Fernet-encrypted; logs, metrics and audit lines carry no question or answer text. Release-blocking tests probe prompts, logs, memory, embeddings, outputs and the agent trace.

Q34. How do you defend against prompt injection? Input: injection and jailbreak patterns are blocked. Documents and web results: instruction-like lines are neutralised and the chunk is flagged and labelled untrusted in the prompt. Output: prompt leaks are blocked. And because every answer is verified against passages, an injected instruction cannot produce an unchecked dose or diagnosis.

Q35. How is the frontend secured? Strict CSP (`script-src 'self'` , no inline script or style, no external assets, `frame-ancestors 'none'`), model output only through `textContent` (ESLint rule forbids `innerHTML`), no-store for API responses.

Q36. And the runtime? Non-root uid 10001, read-only root filesystem, all capabilities dropped, Pod Security "restricted", default-deny NetworkPolicies (backend reachable only from the frontend and monitoring; Redis only from the backend), secrets injected by CD, Trivy gate on fixable HIGH/CRITICAL.

---

8. LLM providers and resilience

Q37. Which models and why? OpenAI-compatible providers through one adapter: EURI `gpt-4.1-mini` primary, then Gemini, then Groq `gpt-oss-120b`; EURI `text-embedding-3-small` (1536 dims). Answers at temperature 0.1-0.3; classifiers, re-ranker and judges at 0. Reasoning models run with `reasoning_effort="low"`.

Q38. What happens when a provider fails? A fallback chain with a circuit breaker per model (3 failures -> open for 30 s, then one trial), retries with backoff only for transient errors. Quota (including gateways that report it as HTTP 403) maps to 429 with Retry-After, provider errors to 502, unavailability to 503. Classifiers fail open to their rules; the verifier fails closed.

Q38b. Is there an LLM gateway? How are models routed and tokens limited? Yes, in-process (`backend/app/rag/gateway.py`, ADR-006). Every model call goes through `gateway.for_task(task)`. A YAML routing table sends triage, scope, rewrite and re-rank to a "fast" chain (`LLM_FAST_CHAIN`, defaulting to the answer chain) and the answer and safety judge to the "quality" chain. Each task has min/max output tokens (the minimum matters because reasoning models return nothing with a tiny budget) and a max input budget. Over the answer budget, the lowest-ranked passages are dropped and the verifier checks exactly what the model saw. Over budget elsewhere behaves like a failed call: rules decide triage and scope, and the judge fails closed. Provider-reported tokens are recorded per task and model. Not yet measured live, so I claim no cost savings.

Q39. How do you manage cost and quotas? An answer cache (only clean, first-turn, non-emergency answers; invalidated when documents change), an LLM prompt cache that only stores plain text, a query-embedding cache keyed by a hash (vectors only), task routing so classifiers can use a cheaper model, per-task token limits, per-user rate limits, re-ranking only 12 candidates, and I always estimate and announce the number of live calls before a live run.

---

9. Evaluation and testing

Q40. How do you test an LLM application offline? Fakes for every external dependency. The fake chat model routes by prompt type and records every prompt, which lets tests assert that PHI never reaches a model. The same pipeline code runs in tests and production.

Q41. What does the safety suite cover? Emergency vignettes, dosing, diagnosis, unsafe requests, injection (including inside uploads and web results), PHI leak probes and out-of-corpus questions - each with a 100% release gate. 240 tests, run in CI on every push, including against an adversarial fake model.

Q42. How do you measure quality? A golden set of 14 questions with verbatim evidence quotes (verified by tests), retrieval metrics (evidence hit rate, MRR), an LLM judge for correctness and citation accuracy, and RAGAS-style faithfulness, answer relevancy, context precision and recall computed in-app. Runs are logged to MLflow from a separate environment. Live run (2026-10-15, offline evaluation on a synthetic test set): safety suite 100% in every set; correctness 0.857, citation accuracy 0.929, faithfulness 1.0, relevancy 0.854. The judge is Groq gpt-oss-120b, a different model family from the answer model, so the model does not grade itself.

Q42b. Did you use RAGAS and LLM-as-judge? Did they agree? Both. The in-app metrics follow RAGAS definitions. As an independent check, I ran the real `ragas` library (0.4.3) in its own environment, because it pins conflicting `openai` versions, on the saved answers. It agreed on context precision and recall (1.0). It was stricter on faithfulness (0.85 vs 1.0): one fair catch, a year taken from a passage header, and one false negative. The run also found bugs in my judge: replies were cut off by tight token limits and scored 0. I fixed that, counted judge errors per run, and re-judged the saved answers. Lesson: evaluate the evaluator.

Q43. Why not use the RAGAS library directly? Its pinned `openai` version conflicts with the app's; installing it would change the app's dependencies. I compute equivalent metrics in-app and keep evaluation tooling in a separate virtual environment.

Q44. How is the UI tested? Playwright drives the installed Microsoft Edge against a demo server that runs the real app with a fake model. 10 tests cover sign-in, cited answers, emergency first, dosing refusal, injection, role gating, report download, mobile layout and the agent trace. The tests caught real bugs: views not hidden because a CSS rule overrode `hidden`, the mobile drawer covering the disclaimer, and trace rows hidden by a `.skip` class collision.

Q45. How do you keep evaluation numbers honest? They are always labelled "offline evaluation on a synthetic test set", never presented as clinical performance, and the docs separate implemented, verified-live and planned capabilities.

---

10. Observability and operations

Q46. What do you monitor? p50/p95/p99 latency per route template, model and retrieval latency, triage events, scope decisions, refusals by reason, guardrail events, hallucination flags, cache hits, circuit state, fallbacks and rate limits - all with bounded labels and no PHI. Grafana dashboard and alert rules (triage spikes, refusal-rate shifts, safety-check failures, open circuits, high latency) are written as code; not yet running in an environment.

Q47. How do you audit? One audit line per request and per action: who, what, outcome, failed checks - never question or answer text; switchable by config.

Q48. Health checks? Liveness never checks dependencies (a slow database must not restart pods); readiness checks the database.

---

11. Delivery and deployment

Q49. How would you deploy it? Two images (backend, frontend) to ECR, EKS in ap-southeast-2 next to the Supabase database, ALB with TLS, HPA/PDB, Redis without persistence, KMS-encrypted secrets, GitHub OIDC for CD. Terraform for VPC, EKS, ECR and OIDC passes `terraform validate`; it has never been applied - that requires explicit approval.

Q50. What does CI do? Lint, backend tests, the safety suite as a separate gate, frontend ESLint + Playwright, both image builds, a smoke test through nginx against a pgvector service with production settings, and a Trivy scan. Actions are pinned to tags I verified exist (one planned tag did not, so I corrected it). The pipeline has not run yet because the repository is not on GitHub.

Q51. Why is the frontend a separate nginx container? Same origin for UI and API (strict CSP, no CORS), backend internals (`/metrics`, `/docs`) are not exposed publicly, and the backend can be locked down so only the frontend and monitoring reach it.

---

12. Trade-offs and judgement

Q52. What is the biggest trade-off you made? Strictness versus helpfulness. Strict grounding means more refusals - live, a dosing question returned the strict refusal instead of the quote-only answer. In this domain a refusal is the right failure mode, but I track refusal rates and would tune prompts with live evaluation.

Q53. Why plain JavaScript instead of React? A small, auditable UI with no build step, no runtime dependencies and a strict CSP. The cost is more manual DOM code; ESLint enforces the no-HTML-injection rule.

Q54. Why Postgres for vectors instead of a dedicated vector database? One store for vectors, full-text search, metadata filters and app tables, with SQL-level owner scoping. At this scale pgvector is sufficient and simpler to operate.

Q55. Why not fine-tune a medical model? Fine-tuning does not give traceable citations, needs curated labelled data and still hallucinates. Retrieval plus verification gives evidence a reviewer can check, and the corpus can be updated without retraining.

Q56. What would you do differently? Run classifiers in parallel to cut latency; build an NER-based PHI detector alongside the patterns; run the live evaluation earlier; and get the CI pipeline running from day one.

---

13. Curveball questions

Q57. Is this clinically validated? What is its accuracy? No, and I do not claim any accuracy. The offline metrics are on a synthetic test set. Before real use it needs clinician sign-off of the triage rules, licensed sources, a documented clinical evaluation and clinician review of sampled answers.

Q58. What if the model ignores your instructions? Then the deterministic checks catch it: uncited sentences, invented numbers, unquoted doses and diagnostic phrasing all replace the answer with a refusal. The emergency notice and disclaimers are never written by the model.

Q59. Could a user disable the safety layer? No. Request fields such as filters, research mode or conversation id only narrow retrieval or select history; tests send extra fields like `"safety": false` and confirm triage still runs.

Q60. What is the most important bug you found? The model-written limitations line bypassing the verifier - found by the safety-review agent, not by my own tests. It showed the value of an independent reviewer; it is now covered by release-blocking tests.

Q61. How long does a question take and how many model calls? About 3-5 chat calls (triage, scope, re-rank, answer, plus the judge on high-risk topics) and one embedding call. Latency targets (p95 <= 15 s) are defined but have not been measured under load yet.

---

14. Two-minute walkthrough (for "tell me about a project you built")

"I built a medical information assistant where safety is the architecture, not a feature. A question first goes through local PHI redaction and deterministic plus LLM triage, so emergencies are answered first with text built in code. A scope agent decides whether it is information, research, diagnosis, dosing, unsafe or out of scope, each with a fixed behaviour. Retrieval is hybrid pgvector plus BM25 in Postgres, fused and re-ranked; the answer agent may only use numbered passages and must cite every sentence; a verifier checks citations, numbers, drug names and verbatim dose quotes, with a temperature-0 judge on high-risk topics that fails closed. Users can see every agent step in a trace panel. It is covered by 419 offline tests - 240 of them a release-blocking safety suite - and an independent safety-review agent found real gaps that I fixed and pinned with tests. I verified the core path live against Supabase and EURI. The trade-off is that it refuses more often than a general chatbot; in this domain that is the right failure mode."